Quantum Kernels & QSVMs — Detailed Notes (Session 10)

Course: CS490/5590 — Quantum Computing Applications in Data Science, AI, & Deep Learning

Instructor: Luke Miller

0) Introduction

Goal. Use **quantum feature-map circuits** to embed classical data into a high-dimensional Hilbert space and compute a **kernel** that a classical **Support Vector Machine (SVM)** can train on.

- A **data point** is $x \in \mathbb{R}^d$ where d is the number of input features (a positive integer).
- A **feature map** is a rule $\varphi : \mathbb{R}^d \to \mathcal{H}$ that sends x to a vector $\varphi(x)$ in a Hilbert space $\mathcal{H}$ (a vector space with an inner product).
- A **kernel** is $K(x, x') = \langle \varphi(x), \varphi(x') \rangle$ where $\langle \cdot, \cdot \rangle$ is the inner product (a real scalar for the kernels we build).
- In the **quantum** setting we prepare an n-qubit state $|\varphi(x)\rangle$ where n is the number of qubits (a positive integer), then set

$$K(x, x') = |\langle \varphi(x) \,|\, \varphi(x') \rangle|^2,$$

a real number in $[0, 1]$.

Figure 0.1 — Big picture (hybrid kernel learning)

```rust
Data x ----> Quantum Feature Map Uϕ(x) ----> K(xi, xj) ---->  SVM (classical)  ----> Predictions
                 |0...0>  ->  |ϕ(x)>
      (n qubits)           (state vector)
```

Symbols in the figure: $U_\varphi(x)$ is a unitary circuit depending on input x; $|0\ldots0\rangle$ is the all-zero basis state; $|\varphi(x)\rangle$ is the prepared state; $K(\cdot,\cdot)$ is the real kernel value.

0.5) Quick recap: VQE and why it appears here

Variational Quantum Eigensolver (VQE). Prepare $|\psi(\theta)\rangle$ with parameters $\theta \in \mathbb{R}^m$ (a real vector of length m) and minimise $E(\theta) = \langle\psi(\theta)|H|\psi(\theta)\rangle$ for a Hamiltonian H (a Hermitian operator).

Why mention VQE. The same circuit tools—parameterised rotations, entanglers, Pauli-based measurement—power quantum feature maps.

Figure 0.2 — VQE vs Kernel workflow

```rust
VQE:  U(θ) -> measure ⟨Pi⟩ -> E(θ) -> Optimizer(θ)
Kern: Uϕ(x) + Uϕ(x') -> overlap -> K(x,x') -> SVM
```

Symbols in the figure: $U(\theta)$ is a parameterised circuit; $\langle P_i \rangle$ are real expectation values of Pauli strings P_i; $E(\theta)$ is a real energy; $U_\varphi(\cdot)$ is a data-dependent circuit.

1) Classical kernel refresher (SVM dual)

Feature map. $\varphi : \mathbb{R}^d \to \mathcal{H}$.

Kernel. $K(x, x') = \langle \varphi(x), \varphi(x') \rangle$ (real scalar).

SVM dual (hard margin). Given (x_i, y_i) with $y_i \in \{-1, +1\}$ and $i = 1, \ldots, N$ (integers), solve

$$\max_{\alpha \in \mathbb{R}^N} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j \, y_i y_j \, K(x_i, x_j) \quad \text{s.t. } \alpha_i \geq 0, \; \sum_i \alpha_i y_i = 0.$$

Figure 1.1 — Kernel trick geometry

```java
Input space (R^d) --ϕ--> Feature space (H)
        Not linearly            Linearly
         separable              separable
```

Symbols in the figure: φ is the feature map; $\mathcal{H}$ is the Hilbert space; linear separability is decided by the SVM using only K.

2) Quantum feature maps

2.1 Definition

$$|\varphi(x)\rangle = U_\varphi(x)\,|0\rangle^{\otimes n}.$$

- $U_\varphi(x)$ is a data-dependent unitary (matrix with $U^\dagger U = I$).
- $|0\rangle^{\otimes n}$ is the n-qubit all-zero state.
- $|\varphi(x)\rangle$ is a unit vector in dimension 2^n.

Figure 2.1 — Generic feature-map layer

```
css

q0: ──RZ(f1(x))──■──────────
                 |
q1: ──RZ(f2(x))──■──RZZ(g12(x))──
                 |
q2: ──RZ(f3(x))──■──────────
```

Symbols in the figure: $RZ(\phi) = e^{-i\phi Z/2}$ with real angle ϕ; $RZZ(\phi) = e^{-i\phi Z\otimes Z/2}$; $f_k(\cdot)$, $g_{ij}(\cdot)$ are real functions of components of x.

2.2 Examples

ZZFeatureMap. Layers of $R_Z(x_i)$ and $R_{ZZ}(x_i x_j)$.

Pauli FeatureMap. $U_\varphi(x) = \exp(-i \sum_k f_k(x) P_k)$ where P_k are Pauli strings.

ZFeatureMap. Only $R_Z(x_i)$ rotations.

Figure 2.2 — ZZFeatureMap (n=3, one layer)

Symbols in the figure: x_0, x_1, x_2 are real features; each RZZ uses the real product $x_i x_j$.

Depth note. Depth $\approx$ layers $\times$ (single-qubit + two-qubit gates), typically ≤ 3 on NISQ devices.

3) Quantum kernel

$$K(x, x') = |\langle \varphi(x) \,|\, \varphi(x') \rangle|^2 = \left| \langle 0|^{\otimes n} \, U_\varphi^\dagger(x) \, U_\varphi(x') \, |0\rangle^{\otimes n} \right|^2 .$$

- $K(x, x')$ is a real number in $[0, 1]$.
- $U_\varphi^\dagger(x)$ is the adjoint of $U_\varphi(x)$.

Figure 3.1 — Kernel estimation circuit (overlap test)

```
bash

|0…0> ── Uϕ(x′) ── Uϕ†(x) ── Measure Z…Z
                  (adjoint)
Outcome '00…0' probability  =  K(x, x′)
```

Symbols in the figure: $U_\varphi(x')$ and $U_\varphi^\dagger(x)$ are unitaries; "Measure $Z \ldots Z$" means computational-basis measurement; probability is a real number in $[0, 1]$.

Shot variance. With M shots (integer), variance is $K(1 - K)/M$ (real).

4) QSVM workflow

1. **Choose feature map** $U_\varphi(x)$.
2. **Estimate kernel matrix** $K_{ij} = K(x_i, x_j)$ for N training points (integers).
3. **Train SVM** with precomputed K.
4. **Predict** a new x_* using the vector $\big(K(x_*, x_i)\big)$ against support vectors.

Figure 4.1 — Data-to-decision pipeline

```scss
X (N×d) —>  Kernel Estimator  —>  K (N×N)  —>  SVM Solver  —> labels ŷ
              (quantum)                        (classical)
```

Symbols in the figure: X is the design matrix; K is the symmetric kernel matrix; $\hat{y}$ are predicted labels.

5) Qiskit demo (iris binary subset)

- Dataset (X, y) with $X \in \mathbb{R}^{N \times d}$ and $y \in \{-1, +1\}^N$.
- Feature map `ZZFeatureMap(num_qubits)` implements $U_\varphi(x)$ on $n = \text{num\_qubits}$ qubits (integer).
- QuantumKernel builds K by running the estimation circuit.
- SVC uses `kernel=quantum_kernel.evaluate`.

```
X, y = load_iris(return_X_y=True)
X = X[y != 2][:, :2]                # d = 2 (real-valued features)
y = y[y != 2]                       # labels in {-1,+1}
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.3, random_state=0)

num_qubits = 2                      # n = 2 qubits (integer)
feature_map = ZZFeatureMap(num_qubits)
backend = BasicAer.get_backend('statevector_simulator')
quantum_kernel = QuantumKernel(feature_map=feature_map, quantum_instance=backend)

svm = SVC(kernel=quantum_kernel.evaluate)
svm.fit(X_train, y_train)
pred = svm.predict(X_test)
print("QSVM accuracy:", accuracy_score(y_test, pred))
```

6) Performance, noise, and scaling

Two-qubit gate errors. Reduce kernel fidelity. **Mitigation:** dynamical decoupling, error-aware transpilation, ZNE.

Shot noise. Variance $K(1-K)/M$. **Mitigation:** variance-aware shot allocation, batching.

Quadratic scaling. $O(N^2)$ circuits. **Mitigation:** subsampling, Nyström/low-rank approximations.

Effective dimension. Overfitting risk. **Mitigation:** SVM regularisation $C > 0$ (real), limit depth.

Figure 6.1 — Where time goes

mathematica

Copy code

```
Kernel Estimation (O(N^2) pairs) : ████████████
SVM Fit (classical)              : ██
I/O & Transpilation              : ▌
```

7) AI / DS applications and open questions

- **Small-N, non-linear** problems benefit from expressive kernels.
- **Scientific signals** with periodic structure align with phase-rotation maps.
- **Open questions:** which data distributions yield kernels that are **hard to approximate classically** yet **robust to noise**?

Figure 7.1 — When kernels help

Symbols in the figure: Circles and dots are class labels; "margin" is the SVM geometric margin (a real distance).

9) FAQ

- **Do we optimise feature-map parameters?** QSVM training is classical; scalar scalings inside U_φ can be cross-validated like hyperparameters.
- **Why squared overlap?** Guarantees a positive semidefinite kernel and equals a measurable probability.
- **Can shots be reused?** Each pair (x_i, x_j) uses its own $U_\varphi^\dagger(x_i)U_\varphi(x_j)$; batching reduces overhead.
- **How to handle multiclass?** One-vs-rest SVMs or multiclass SVM formulations.

10) Summary (Session 10)

- **What:** Build $K(x, x') = |\langle \varphi(x) \,|\, \varphi(x') \rangle|^2$ via a feature-map circuit $U_\varphi(x)$.
- **How:** Estimate the kernel matrix on quantum hardware; train a classical SVM.
- **Why:** Entangling feature maps create non-linear similarities in 2^n-dimensional space with few qubits; SVMs convert similarities to decision boundaries.
- **Caveats:** Noise and $O(N^2)$ scaling; use mitigation and approximations.